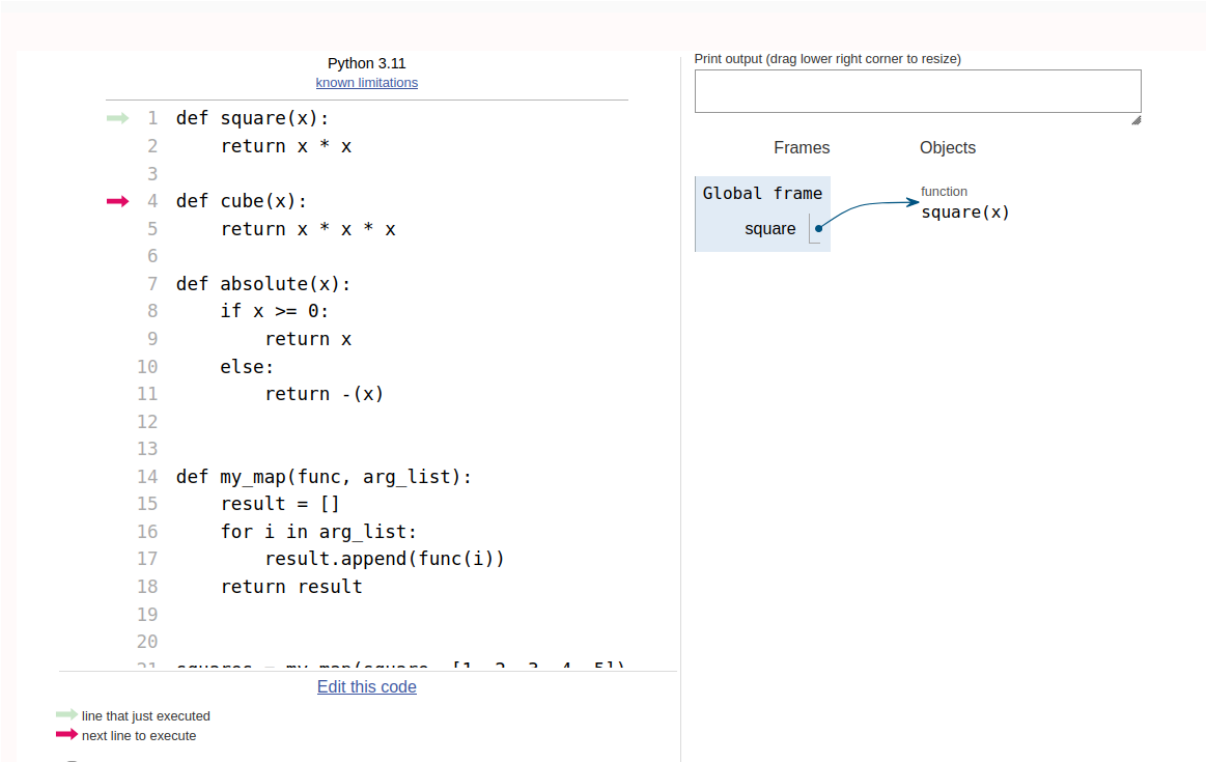
```python
def square(x):
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
print(squares)
```



Python 3.11
known limitations

```
def square(x):
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
```

Edit this code

line that just executed
next line to execute

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)

Python 3.11
known limitations

```
def square(x):
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 4 of 36

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)
- absolute → function absolute(x)

Python 3.11

known limitations

```
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
print(squares)
```

Edit this code

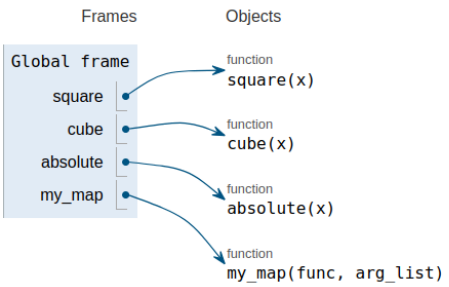
line that just executed

next line to execute

<< First | < Prev | Next > | Last >>

Step 5 of 36

Print output (drag lower right corner to resize)



Python 3.11

known limitations

```
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
print(squares)
```

Edit this code

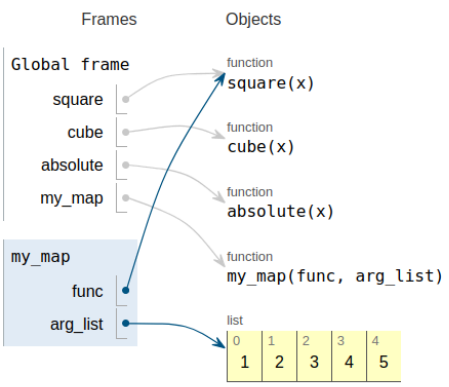
line that just executed

next line to execute

<< First | < Prev | Next > | Last >>

Step 6 of 36

Print output (drag lower right corner to resize)



Python 3.11
known limitations

```
1  def square(x):
2      return x * x
3
4  def cube(x):
5      return x * x * x
6
7  def absolute(x):
8      if x >= 0:
9          return x
10     else:
11         return -(x)
12
13
14 def my_map(func, arg_list):
15     result = []
16     for i in arg_list:
17         result.append(func(i))
18     return result
19
20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 12 of 36

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)
- absolute → function absolute(x)
- my_map → function my_map(func, arg_list)

my_map
- func → function square(x)
- arg_list → list [0: 1, 1: 2, 2: 3, 3: 4, 4: 5]
- result → empty list
- i: 1

square
- x: 1
- Return value: 1

Python 3.11
known limitations

```
1  def square(x):
2      return x * x
3
4  def cube(x):
5      return x * x * x
6
7  def absolute(x):
8      if x >= 0:
9          return x
10     else:
11         return -(x)
12
13
14 def my_map(func, arg_list):
15     result = []
16     for i in arg_list:
17         result.append(func(i))
18     return result
19
20
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 32 of 36

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)
- absolute → function absolute(x)
- my_map → function my_map(func, arg_list)

my_map
- func → function square(x)
- arg_list → list [0: 1, 1: 2, 2: 3, 3: 4, 4: 5]
- result → list [0: 1, 1: 4, 2: 9, 3: 16]
- i: 5

square
- x: 5
- Return value: 25

Python 3.11
known limitations

```
1  def square(x):
2      return x * x
3
4  def cube(x):
5      return x * x * x
6
7  def absolute(x):
8      if x >= 0:
9          return x
10     else:
11         return -(x)
12
13
14 def my_map(func, arg_list):
15     result = []
16     for i in arg_list:
17         result.append(func(i))
18     return result
19
20
```

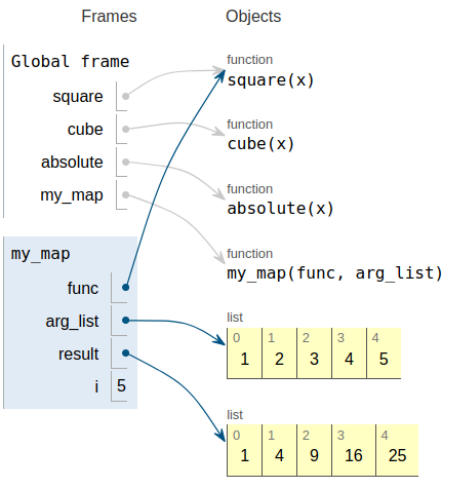
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 33 of 36

Print output (drag lower right corner to resize)



Python 3.11
known limitations

```
1  def square(x):
2      return x * x
3
4  def cube(x):
5      return x * x * x
6
7  def absolute(x):
8      if x >= 0:
9          return x
10     else:
11         return -(x)
12
13
14 def my_map(func, arg_list):
15     result = []
16     for i in arg_list:
17         result.append(func(i))
18     return result
19
20
```

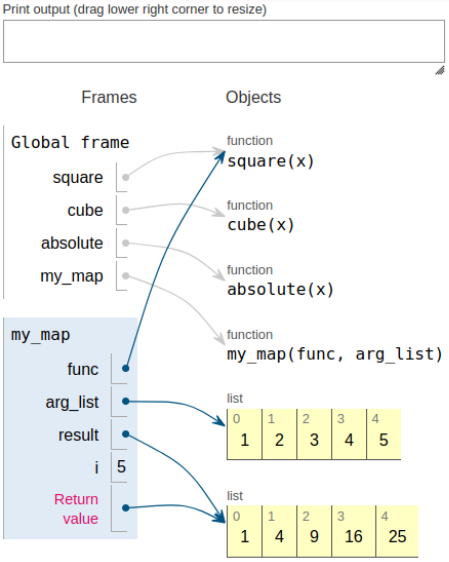
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 35 of 36

Print output (drag lower right corner to resize)



Python 3.11
known limitations

```
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
print(squares)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 36 of 36

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)
- absolute → function absolute(x)
- my_map → function my_map(func, arg_list)
- squares → list [0: 1, 1: 4, 2: 9, 3: 16, 4: 25]

Python 3.11
known limitations

```
    return x * x

def cube(x):
    return x * x * x

def absolute(x):
    if x >= 0:
        return x
    else:
        return -(x)


def my_map(func, arg_list):
    result = []
    for i in arg_list:
        result.append(func(i))
    return result


squares = my_map(square, [1, 2, 3, 4, 5])
print(squares)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (36 steps)

Print output (drag lower right corner to resize)

```
[1, 4, 9, 16, 25]
```

Frames | Objects

Global frame
- square → function square(x)
- cube → function cube(x)
- absolute → function absolute(x)
- my_map → function my_map(func, arg_list)
- squares → list [0: 1, 1: 4, 2: 9, 3: 16, 4: 25]

Fase 1: Inicialización del Espacio de Nombres Global (Frames Globales)

(Corresponde a las primeras páginas del PDF, Steps iniciales hasta el Step 5)

+1

1. **Definición de Funciones como Objetos de Primera Clase:**
    - El intérprete lee las sentencias `def square(x):`, `def cube(x):`, `def absolute(x):` y `def my_map(func, arg_list):`.
      +3
    - La palabra clave `def` es una instrucción ejecutable, no solo una declaración. Al ejecutarla, CPython reserva memoria en el **Heap** (la columna *Objects*) para crear estructuras de datos de tipo `function`. Estas estructuras de memoria de C (llamadas `PyFunctionObject`) contienen el *bytecode* compilado de la función, sus constantes locales y su entorno de cierre (closure).
      +1
    - Simultáneamente, en el **Global Frame** (que es técnicamente el diccionario devuelto por la función `globals()`), el intérprete inserta cadenas de texto con los nombres de las funciones (`"square"`, `"cube"`, etc.) como claves.
    - El valor asignado a cada una de estas claves es la **dirección de memoria hexadecimal** que apunta al objeto `PyFunctionObject` correspondiente recién creado en el Heap.
      +1

Fase 2: Evaluación de Argumentos e Invocación (Step 5 a 6)

(Página 3 del PDF)

+1

2. **Preparación de la Llamada Principal:**
    - El puntero de instrucción llega a la línea: `squares = my_map(square, [1, 2, 3, 4, 5])`. Antes de poder asignar nada a la variable `squares`, el lado derecho de la igualdad debe evaluarse.
    - **Evaluación del Literal de Lista:** Python detecta el literal `[1, 2, 3, 4, 5]`. Inmediatamente interactúa con el asignador de memoria del sistema operativo para reservar un bloque en el Heap para un nuevo objeto de tipo `list` (un array dinámico en C, llamado `PyListObject`). Se inicializa con referencias a los objetos enteros del 1 al 5.
      +4
    - **Paso de Referencias:** El intérprete localiza la dirección de memoria de `square` en el diccionario Global.
3. **Creación del Frame de Ejecución (`my_map`):**
    - Se efectúa la llamada a la función. CPython empuja (pushes) un nuevo *Frame* (marco de pila) llamado `my_map` en la parte superior del **Call Stack**. Este frame contiene su propio diccionario de ámbito local (`locals()`).

- **Paso por Asignación (Pass-by-Assignment):** En este nuevo diccionario local, se crean dos identificadores: `func` y `arg_list`. El intérprete copia la *dirección de memoria* del objeto función global y se la asigna a `func`. Copia la *dirección de memoria* del nuevo objeto lista del Heap y se la asigna a `arg_list`. Por esto ves las flechas cruzando desde el Frame local hacia los objetos compartidos en la memoria dinámica.
  +4

Fase 3: El Bucle y la Resolución de Pila (Steps 12 a 32)

(Página 4 del PDF)

+1

4. **Estado Interno Avanzado:**
   - En el frame de `my_map`, se ejecuta `result = []`. Se asigna un nuevo objeto lista vacío en el Heap, y su dirección de memoria se vincula a la variable local `result`.
     +2
   - **El Clímax de la Pila (Step 32):** El PDF captura el momento exacto de la última iteración del bucle `for`, evaluando `func(i)` donde `i` se ha vinculado a la dirección de memoria del objeto entero `5`.
     +2
   - Se ha empujado un tercer frame a la pila: `square`. El identificador local `x` apunta a la dirección de memoria del entero `5`.
     +1
   - El procesador calcula `5 * 5`, CPython aloja un nuevo objeto entero `25` en el Heap, y guarda temporalmente su dirección en el registro especial del frame local llamado `Return value`.

Fase 4: Desenrollado de la Pila y Mutación del Objeto (Steps 33 a 35)

(Página 5 del PDF)

+1

5. **Destrucción del Frame Interno:**
   - Al ejecutar el `return`, el frame superior (`square`) se elimina del Call Stack (se le hace un *pop* y se destruye su diccionario de ámbito local). La dirección de memoria del entero `25` sobrevive y se devuelve al frame subyacente (`my_map`).
     +1
6. **Mutación *In-Place*:**
   - El método `.append()` toma esa dirección de memoria de `25` y reasigna el array interno en C del objeto lista objetivo (`result`), agregando el nuevo puntero al final del bloque continuo de memoria. El objeto lista en el Heap

crece: `[1, 4, 9, 16, 25]`.
+1

7. **Preparación del Retorno Final:**
   - El bucle `for` concluye. Se ejecuta la línea `return result`. El frame de `my_map` establece la dirección de memoria de esta lista poblada en su registro `Return value`.
     +2

Fase 5: Asignación Global y Garbage Collection (Step 36 y Fin)

(Página 6 del PDF)

+1

8. **Limpieza y Regreso al Ámbito Global:**
   - El frame de `my_map` sufre un *pop* y se destruye.
   - **Resolución de la Asignación Inicial:** La dirección de memoria que retornó la función finalmente se procesa en la línea original: `squares = my_map(...)`. El Global Frame crea la clave `"squares"` y le asigna el puntero al objeto lista `[1, 4, 9, 16, 25]` que reside en el Heap.
     +1
9. **Salida a Sistema (`stdout`):**
   - La línea `print(squares)` pasa esta dirección de memoria a la función de C subyacente para imprimir. Python busca la representación en texto del objeto lista y la envía al canal de salida estándar del sistema operativo (mostrado en "Print output"): `[1, 4, 9, 16, 25]`.
     +1
10. **Término del Proceso (Done running):**
    - El script finaliza. CPython inicia la limpieza final. Las referencias globales se destruyen, los contadores de referencia (Reference Counting) de todos los objetos en el Heap caen a cero. El recolector de basura (Garbage Collector) recupera todos los bloques de memoria dinámica y los devuelve a la tabla de páginas del sistema operativo.

Esta es la anatomía técnica real de la ejecución. Cada línea de código es simplemente una directriz sobre cómo manipular diccionarios de referencias cruzadas entre un Stack de ejecución secuencial y un montón de objetos (Heap) fragmentado.